



We use optional cookies to improve your experience on our websites, such as through social media connections, and to display personalized advertising based on your online activity. If you reject optional cookies, only cookies necessary to provide you the services will be used. You may change your selection by clicking "Manage Cookies" at the bottom of the page. [Privacy Statement](#) [Third-Party Cookies](#)

Accept

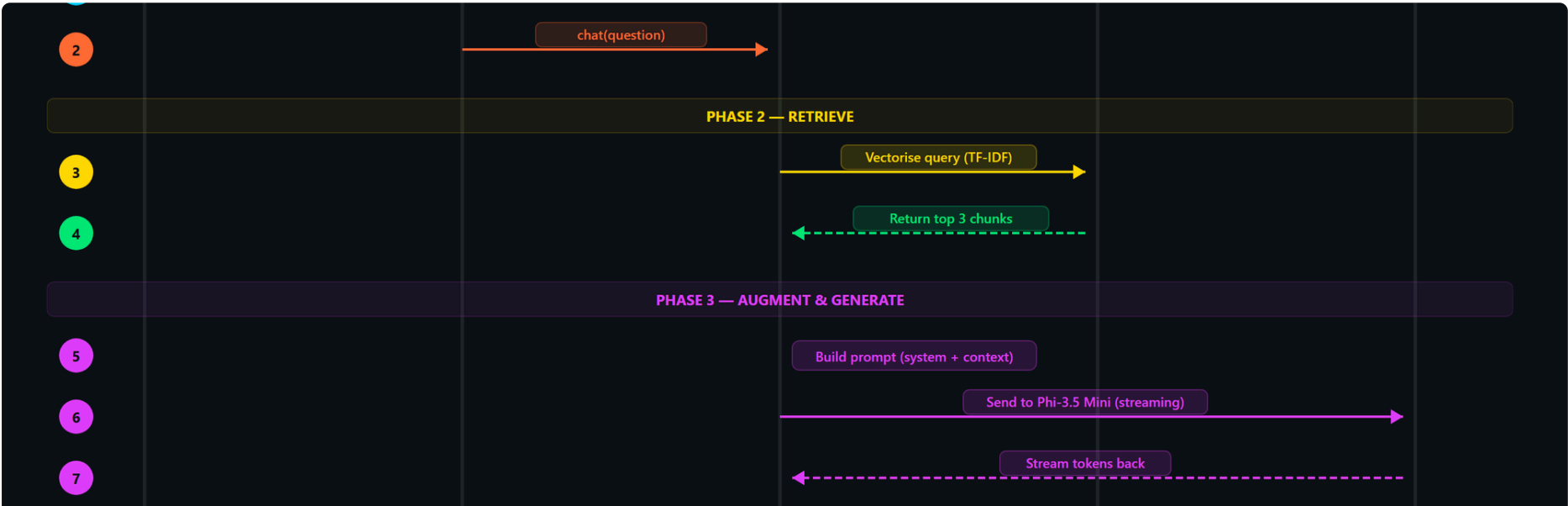
Reject

Manage cookies



Microsoft Developer Community Blog

🔍 Search this community



MICROSOFT DEVELOPER COMMUNITY BLOG 11 MIN READ

Building Your First Local RAG Application with Foundry Local



Lee Stott

 MICROSOFT

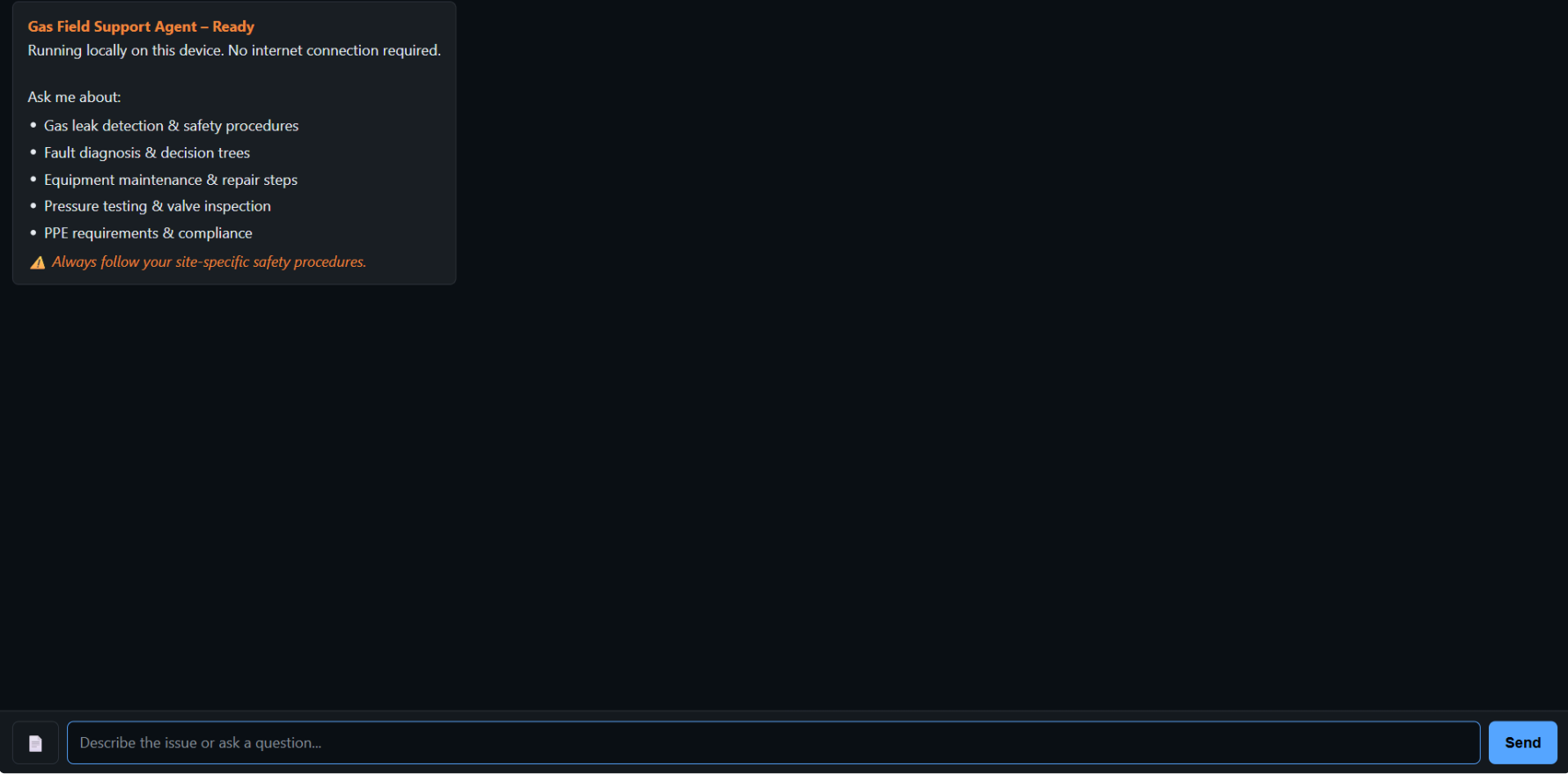
Mar 30, 2026

A developer's guide to building an offline, mobile-responsive AI support agent using Retrieval-Augmented Generation, the Foundry Local SDK, and JavaScript.

Imagine you are a gas field engineer standing beside a pipeline in a remote location. There is no Wi-Fi, no mobile signal, and you need a safety procedure right now. What do you do?

This is the exact problem that inspired this project: a **fully offline RAG-powered support agent** that runs entirely on your machine. No cloud. No API keys. No outbound network calls. Just a local language model, a local vector store, and your own documents, all accessible from a browser on any device.

In this post, you will learn how it works, how to build your own, and the key architectural decisions behind it. If you have ever wanted to build an AI application that runs locally and answers questions grounded in your own data, this is the place to start.



The finished application: a browser-based AI support agent that runs entirely on your machine.

What Is Retrieval-Augmented Generation?

Retrieval-Augmented Generation (RAG) is a pattern that makes AI models genuinely useful for domain-specific tasks. Rather than hoping the model "knows" the answer from its training data, you:

1. **Retrieve** relevant chunks from your own documents using a vector store
2. **Augment** the model's prompt with those chunks as context
3. **Generate** a response grounded in your actual data

The result is fewer hallucinations, traceable answers with source attribution, and an AI that works with *your* content rather than relying on general knowledge.

If you are building internal tools, customer support bots, field manuals, or knowledge bases, RAG is the pattern you want.

RAG vs CAG: Understanding the Trade-offs

If you have explored AI application patterns before, you have likely encountered **Context-Augmented Generation (CAG)**. Both RAG and CAG solve the same core problem: grounding an AI model's answers in your own content. They take different approaches, and each has genuine strengths and limitations.

RAG (Retrieval-Augmented Generation)

How it works: Documents are split into chunks, vectorised, and stored in a database. At query time, the most relevant chunks are retrieved and injected into the prompt.

Strengths:

- Scales to thousands or millions of documents
- Fine-grained retrieval at chunk level with source attribution
- Documents can be added or updated dynamically without restarting
- Token-efficient: only relevant chunks are sent to the model
- Supports runtime document upload via the web UI

Limitations:

- More complex architecture: requires a vector store and chunking strategy
- Retrieval quality depends on chunking parameters and scoring method
- May miss relevant content if the retrieval step does not surface it



How it works: All documents are loaded at startup. The most relevant ones are selected per query using keyword scoring and injected into the prompt.

Strengths:

- Drastically simpler architecture with no vector database or embeddings
- All information is always available to the model
- Minimal dependencies and easy to set up
- Near-instant document selection

Limitations:

- Constrained by the model's context window size
- Best suited to small, curated document sets (tens of documents)
- Adding documents requires an application restart

Want to compare these patterns hands-on? There is a [CAG-based implementation](#) of the same gas field scenario using whole-document context injection. Clone both repositories, run them side by side, and see how the architectures differ in practice.

When Should You Choose Which?

Consideration	Choose RAG	Choose CAG
Document count	Hundreds or thousands	Tens of documents
Document updates	Frequent or dynamic (runtime upload)	Infrequent (restart to reload)
Source attribution	Per-chunk with relevance scores	Per-document
Setup complexity	Moderate (ingestion step required)	Minimal
Query precision	Better for large or diverse collections	Good for keyword-matchable content
Infrastructure	SQLite vector store (single file)	None beyond the runtime

For the sample application in this post (20 gas engineering procedure documents with runtime upload), RAG is the clear winner. If your document set is small and static, CAG may be simpler. Both patterns run fully offline using Foundry Local.

Foundry Local: Your On-Device AI Runtime

[Foundry Local](#) is a lightweight runtime from Microsoft that downloads, manages, and serves language models entirely on your device. No cloud account, no API keys, no outbound network calls (after the initial model download).

What makes it particularly useful for developers:

- **No GPU required:** runs on CPU or NPU, making it accessible on standard laptops and desktops
- **Native SDK bindings:** in-process inference via the `foundry-local-sdk` npm package, with no HTTP round-trips to a local server
- **Automatic model management:** downloads, caches, and loads models automatically
- **Hardware-optimised variant selection:** the SDK picks the best variant for your hardware (GPU, NPU, or CPU)
- **Real-time progress callbacks:** ideal for building loading UIs that show download and initialisation progress

The integration code is refreshingly minimal:



```
3 // Create a manager and discover models via the catalogue
4 const manager = FoundryLocalManager.create({ appName: "gas-field-local-rag" });
5 const model = await manager.catalog.getModel("phi-3.5-mini");
6
7 // Download if not cached, then load into memory
8 if (!model.isCached) {
9   await model.download((progress) => {
10     console.log(`Download: ${Math.round(progress * 100)}%`);
11   });
12 }
13 await model.load();
14
15 // Create a chat client for direct in-process inference
16 const chatClient = model.createChatClient();
17 const response = await chatClient.completeChat([
18   { role: "system", content: "You are a helpful assistant." },
19   { role: "user", content: "How do I detect a gas leak?" }
20 ]);
```

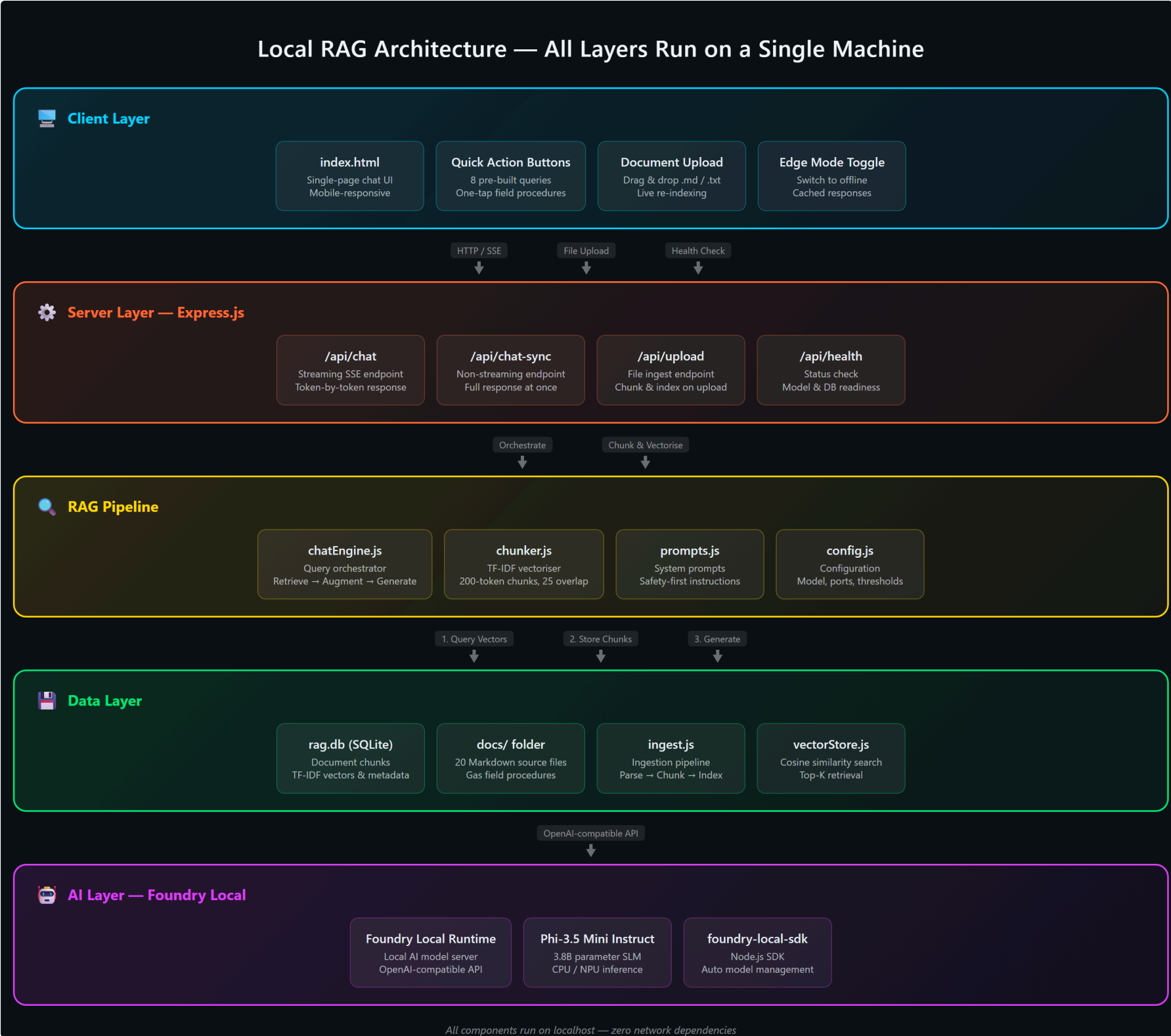
That is it. No server configuration, no authentication tokens, no cloud provisioning. The model runs in the same process as your application.

The Technology Stack

The sample application is deliberately simple. No frameworks, no build steps, no Docker:

Layer	Technology	Purpose
AI Model	Foundry Local + Phi-3.5 Mini	Runs locally via native SDK bindings, no GPU required
Back end	Node.js + Express	Lightweight HTTP server, everyone knows it
Vector Store	SQLite (via better-sqlite3)	Zero infrastructure, single file on disc
Retrieval	TF-IDF + cosine similarity	No embedding model required, fully offline
Front end	Single HTML file with inline CSS	No build step, mobile-responsive, field-ready

The total dependency footprint is three npm packages: [express](#), [foundry-local-sdk](#), and [better-sqlite3](#).



The five-layer architecture, all running on a single machine.

The system has five layers, all running on a single machine:

- **Client layer:** a single HTML file served by Express, with quick-action buttons and a responsive chat interface
- **Server layer:** Express.js starts immediately and serves the UI plus SSE status and chat endpoints
- **RAG pipeline:** the chat engine orchestrates retrieval and generation; the chunker handles TF-IDF vectorisation; the prompts module provides safety-first system instructions
- **Data layer:** SQLite stores document chunks and their TF-IDF vectors; documents live as `.md` files in the `docs/` folder
- **AI layer:** Foundry Local runs Phi-3.5 Mini on CPU or NPU via native SDK bindings

Building the Solution Step by Step

Prerequisites

You need two things installed on your machine:

1. **Node.js 20 or later:** [download from nodejs.org](#)
2. **Foundry Local:** Microsoft's on-device AI runtime:

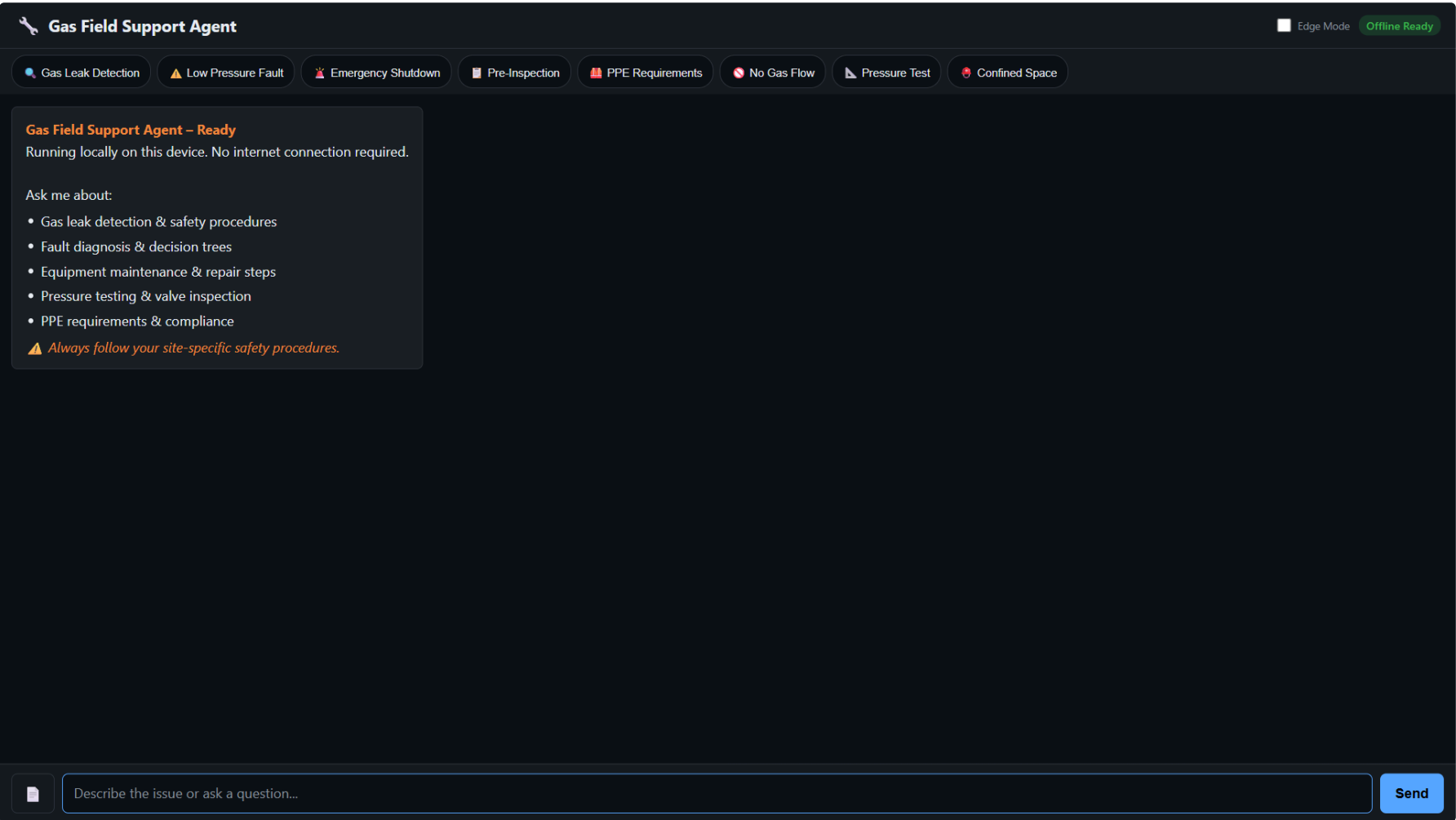
```
winget install Microsoft.FoundryLocal
```

The SDK will automatically download the Phi-3.5 Mini model (approximately 2 GB) the first time you run the application.

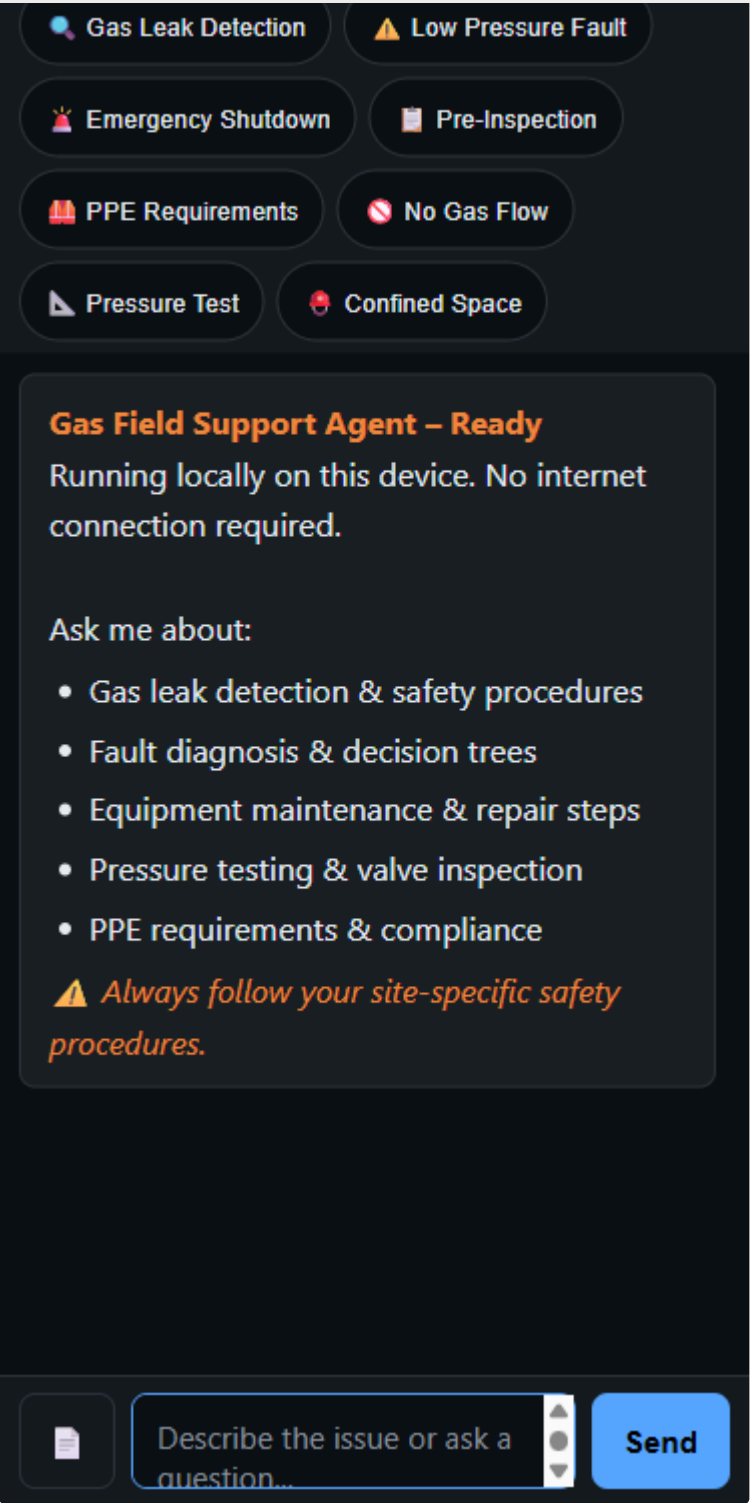


```
1  # Clone the repository
2  git clone https://github.com/leestott/local-rag.git
3  cd local-rag
4
5  # Install dependencies
6  npm install
7
8  # Ingest the 20 gas engineering documents into the vector store
9  npm run ingest
10
11 # Start the server
12 npm start
```

Open **http://127.0.0.1:3000** in your browser. You will see the status indicator whilst the model loads. Once the model is ready, the status changes to "Offline Ready" and you can start chatting.



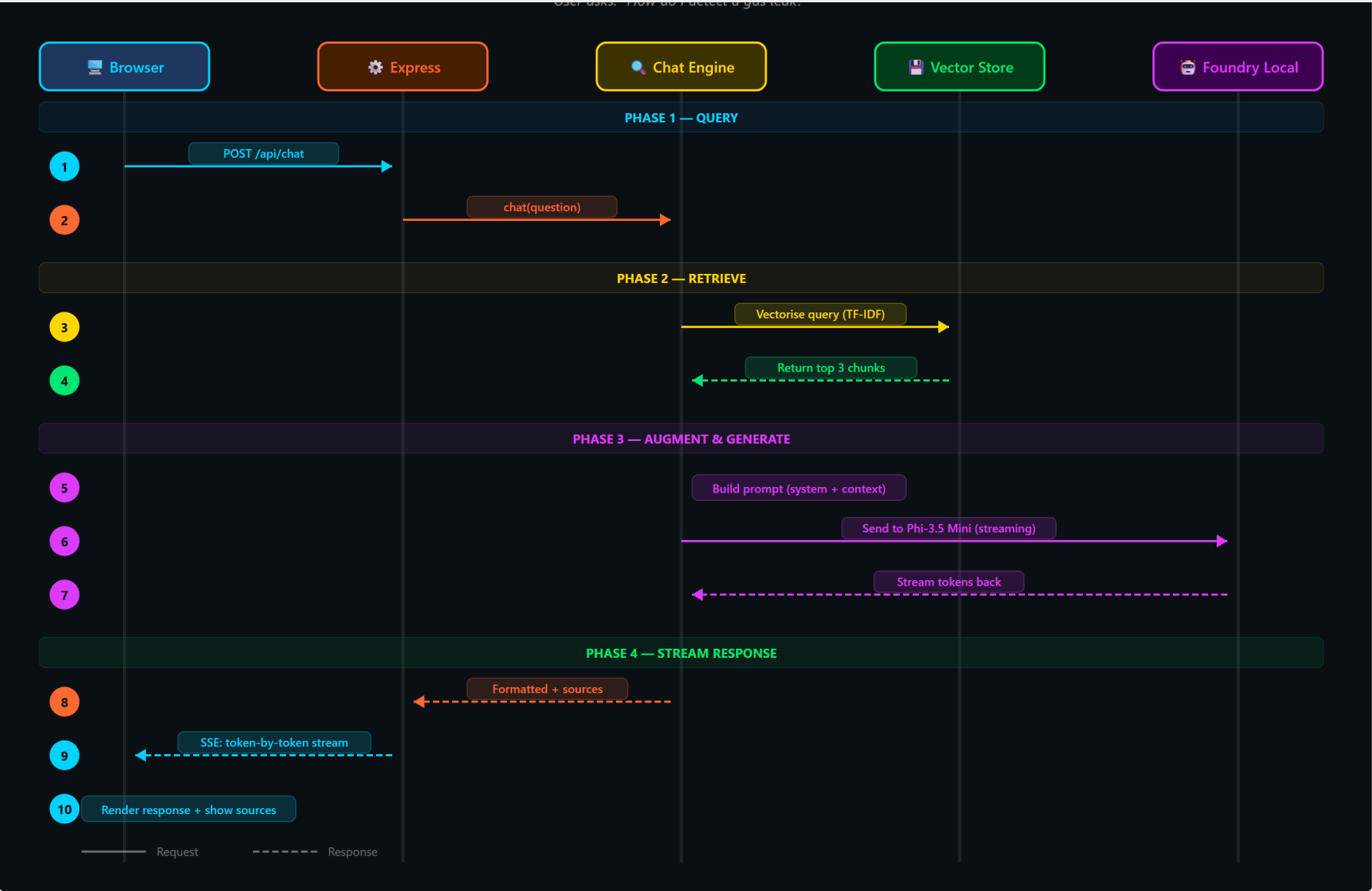
Desktop view



Mobile view

How the RAG Pipeline Works

Let us trace what happens when a user asks: "**How do I detect a gas leak?**"



The query flow from browser to model and back.

1 Documents are ingested and indexed

When you run `npm run ingest`, every `.md` file in the `docs/` folder is read, parsed (with optional YAML front-matter for title, category, and ID), split into overlapping chunks of approximately 200 tokens, and stored in SQLite with TF-IDF vectors.

2 Model is loaded via the SDK

The Foundry Local SDK discovers the model in the local catalogue and loads it into memory. If the model is not already cached, it downloads it first (with progress streamed to the browser via SSE).

3 User sends a question

The question arrives at the Express server. The chat engine converts it into a TF-IDF vector, uses an inverted index to find candidate chunks, and scores them using cosine similarity. The top 3 chunks are returned in under 1 ms.

4 Prompt is constructed

The engine builds a messages array containing: the system prompt (with safety-first instructions), the retrieved chunks as context, the conversation history, and the user's question.

5 Model generates a grounded response

The prompt is sent to the locally loaded model via the Foundry Local SDK's native chat client. The response streams back token by token through Server-Sent Events to the browser. Source references with relevance scores are included.



How do I detect a gas leak?

Summary:

To detect a gas leak, follow a systematic visual inspection and use a calibrated gas detector.

Safety Warnings :

- Eliminate all ignition sources before starting.

- Wear approved gas-rated PPE.

- Maintain a safe distance during detection.

- Evacuate and call emergency control if LEL reading exceeds 20%.

Step-by-step Guidance:

1. Inspect all joints, fittings, and connection points visually.

2. Check the calibration date of your gas detector; do not use an expired one.

3. Power on the gas detector and confirm zero reading in clean air.

4. Slowly move the detector (2–3 cm/s) around suspect areas.

5. If a gas is detected above the threshold:

a. Stop work immediately.

b. Mark the leak location.

c. Isolate the section using valves.

d. Follow emergency protocol GP-EM-01.

6. If the reading is below threshold, apply leak detection fluid and observe for bubble formation for at least 30 seconds.

7. Record all findings.

Reference:

- Gas Field Operations Manual – Section 4.2, British Standard BS 6400 – Leak Detection

Remember to report all confirmed leaks to the control room within 15 minutes and do not attempt repairs until isolation is confirmed and authorised.

Sources (3)

Describe the issue or ask a question...

Send

A response with safety warnings and step-by-step guidance

Gas Field Support Agent

Edge ModeOffline Ready

Gas Leak Detection

Low Pressure Fault

Emergency Shutdown

Pre-Inspection

PPE Requirements

No Gas Flow

Pressure Test

Confined Space

How do I detect a gas leak?

Summary:

To detect a gas leak, follow a systematic visual inspection and use a calibrated gas detector.

Safety Warnings :

- Eliminate all ignition sources before starting.

- Wear approved gas-rated PPE.

- Maintain a safe distance during detection.

- Evacuate and call emergency control if LEL reading exceeds 20%.

Step-by-step Guidance:

1. Inspect all joints, fittings, and connection points visually.

2. Check the calibration date of your gas detector; do not use an expired one.

3. Power on the gas detector and confirm zero reading in clean air.

4. Slowly move the detector (2–3 cm/s) around suspect areas.

5. If a gas is detected above the threshold:

a. Stop work immediately.

b. Mark the leak location.

c. Isolate the section using valves.

d. Follow emergency protocol GP-EM-01.

6. If the reading is below threshold, apply leak detection fluid and observe for bubble formation for at least 30 seconds.

7. Record all findings.

Reference:

- Gas Field Operations Manual – Section 4.2, British Standard BS 6400 – Leak Detection

Remember to report all confirmed leaks to the control room within 15 minutes and do not attempt repairs until isolation is confirmed and authorised.

Sources (3)

Gas Leak Detection – Field Procedure [Inspection Procedures]

29%

Describe the issue or ask a question...

Send

The sources panel shows which chunks were used and their relevance

Key Code Walkthrough

The Vector Store (TF-IDF + SQLite)

The vector store uses SQLite to persist document chunks alongside their TF-IDF vectors. At query time, an inverted index finds candidate chunks that share terms with the query, then cosine similarity ranks them:

https://techcommunity.microsoft.com/blog/azuredevcommunityblog/building-your-first-local-rag-application-with-foundry-local/4501968

9/16



```
3   const queryTf = termFrequency(query);
4   this._ensureCache(); // Build in-memory cache on first access
5
6   // Use inverted index to find candidates sharing at least one term
7   const candidateIndices = new Set();
8   for (const term of queryTf.keys()) {
9     const indices = this._invertedIndex.get(term);
10    if (indices) {
11      for (const idx of indices) candidateIndices.add(idx);
12    }
13  }
14
15  // Score only candidates, not all rows
16  const scored = [];
17  for (const idx of candidateIndices) {
18    const row = this._rowCache[idx];
19    const score = cosineSimilarity(queryTf, row.tf);
20    if (score > 0) scored.push({ ...row, score });
21  }
22
23  scored.sort((a, b) => b.score - a.score);
24  return scored.slice(0, topK);
25 }
```

The inverted index, in-memory row cache, and prepared SQL statements bring retrieval time to sub-millisecond for typical query loads.

Why TF-IDF Instead of Embeddings?

Most RAG tutorials use embedding models for retrieval. This project uses TF-IDF because:

- **Fully offline:** no embedding model to download or run
- **Zero latency:** vectorisation is instantaneous (it is just maths on word frequencies)
- **Good enough:** for 20 domain-specific documents, TF-IDF retrieves the right chunks reliably
- **Transparent:** you can inspect the vocabulary and weights, unlike neural embeddings

For larger collections or when semantic similarity matters more than keyword overlap, you would swap in an embedding model. For this use case, TF-IDF keeps the stack simple and dependency-free.

The System Prompt

For safety-critical domains, the system prompt is engineered to prioritise safety, prevent hallucination, and enforce structured responses:

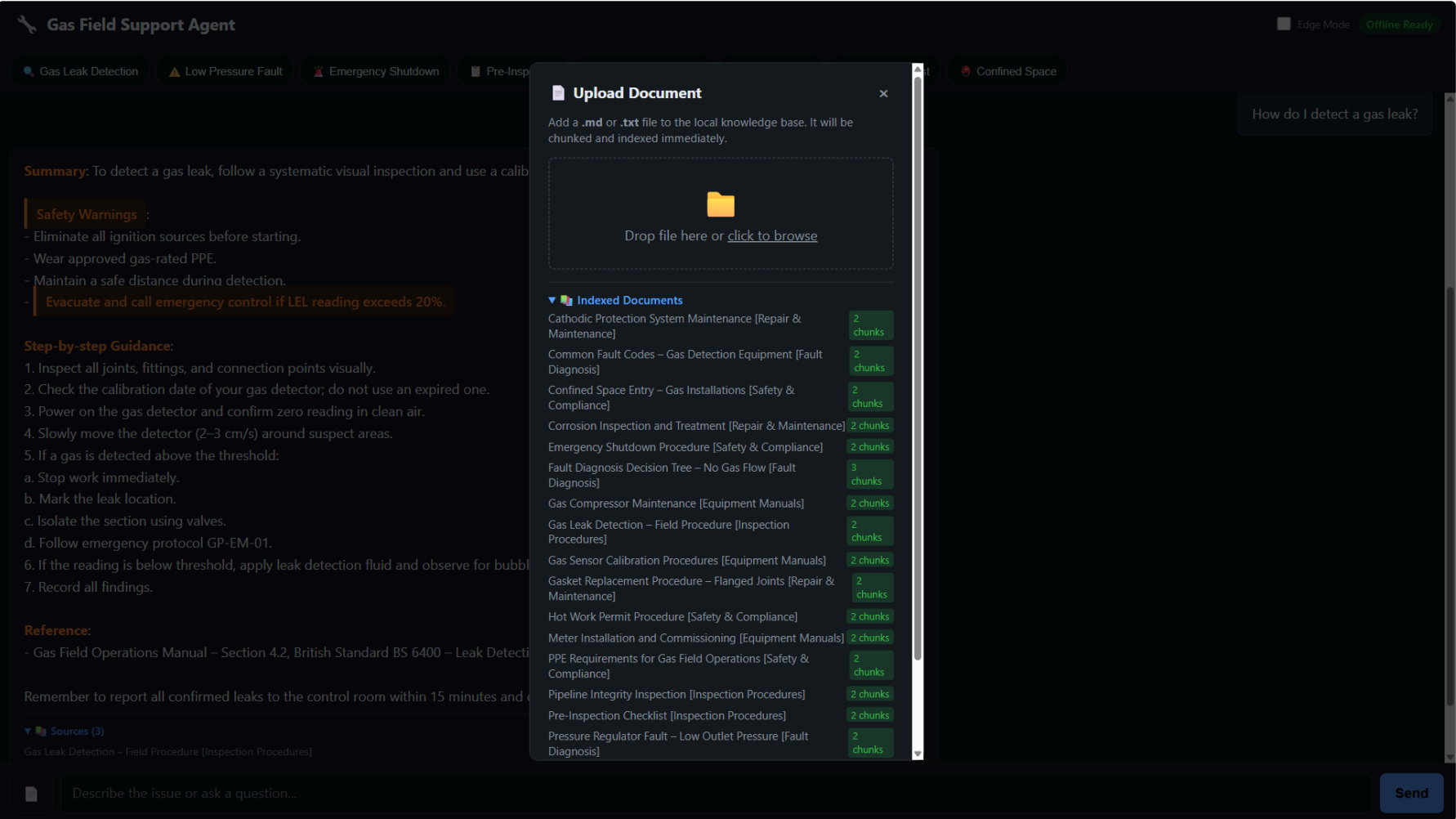


```
3 for gas field inspection and maintenance engineers.
4
5 Behaviour Rules:
6 - Always prioritise safety. If a procedure involves risk,
7   explicitly call it out.
8 - Do not hallucinate procedures, measurements, or tolerances.
9 - If the answer is not in the provided context, say:
10  "This information is not available in the local knowledge base."
11
12 Response Format:
13 - Summary (1-2 lines)
14 - Safety Warnings (if applicable)
15 - Step-by-step Guidance
16 - Reference (document name + section)`;
```

This pattern is transferable to any safety-critical domain: medical devices, electrical work, aviation maintenance, or chemical handling.

Runtime Document Upload

Unlike the CAG approach, RAG supports adding documents without restarting the server. Click the upload button to add new `.md` or `.txt` files. They are chunked, vectorised, and indexed immediately.



The upload modal with the complete list of indexed documents.

Adapting This for Your Own Domain

The sample project is designed to be forked and adapted. Here is how to make it yours in four steps:

1. Replace the documents

Delete the gas engineering documents in `docs/` and add your own markdown files. The ingestion pipeline handles any markdown content with optional YAML front-matter:



```
1  ---
2  title: Troubleshooting Widget Errors
3  category: Support
4  id: KB-001
5  ---
6
7  # Troubleshooting Widget Errors
8  ...your content here...
```

2. Edit the system prompt

Open `src/prompts.js` and rewrite the system prompt for your domain. Keep the structure (summary, safety, steps, reference) and update the language to match your users' expectations.

3. Tune the retrieval

In `src/config.js`:

- `chunkSize: 200`: smaller chunks give more precise retrieval, less context per chunk
- `chunkOverlap: 25`: prevents information falling between chunks
- `topK: 3`: how many chunks to retrieve per query (more gives more context but slower generation)

4. Swap the model

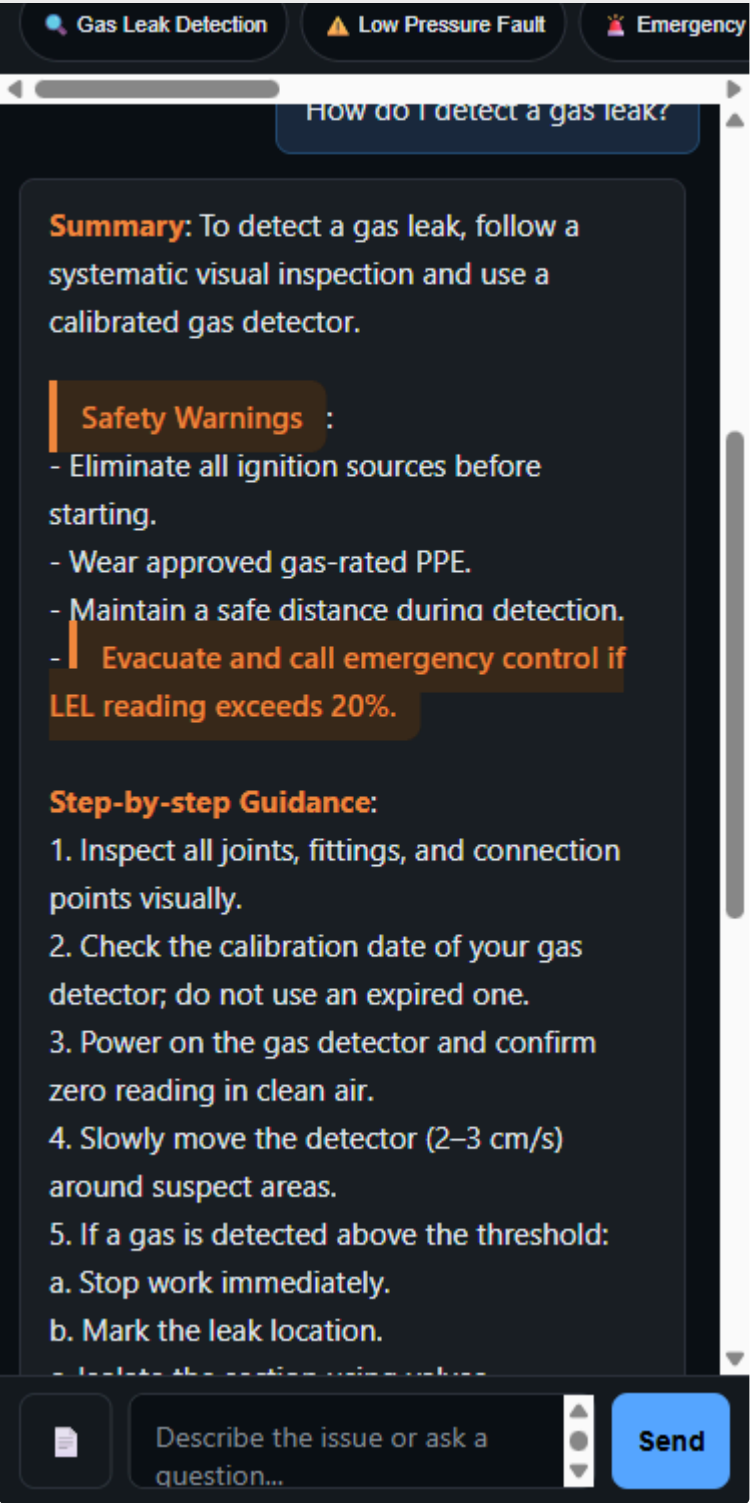
Change `config.model` in `src/config.js` to any model available in the Foundry Local catalogue. Smaller models give faster responses on constrained devices; larger models give better quality.

Building a Field-Ready UI

The front end is a single HTML file with inline CSS. No React, no build tooling, no bundler. This keeps the project accessible to beginners and easy to deploy.

Design decisions that matter for field use:

- **Dark, high-contrast theme** with 18px base font size for readability in bright sunlight
- **Large touch targets** (minimum 44px) for operation with gloves or PPE
- **Quick-action buttons** that wrap on mobile so all options are visible without scrolling
- **Responsive layout** that works from 320px to 1920px+ screen widths
- **Streaming responses** via SSE, so the user sees tokens arriving in real time



The mobile chat experience, optimised for field use.

Testing

The project includes unit tests using the built-in Node.js test runner, with no extra test framework needed:

```
1 | # Run all tests
2 | npm test
```

Tests cover the chunker, vector store, configuration, and server endpoints. Use them as a starting point when you adapt the project for your own domain.

Ideas for Extending the Project

Once you have the basics running, there are plenty of directions to explore:

- **Embedding-based retrieval:** use a local embedding model for better semantic matching on diverse queries
- **Conversation memory:** persist chat history across sessions using local storage or a lightweight database
- **Multi-modal support:** add image-based queries (photographing a fault code, for example)
- **PWA packaging:** make it installable as a standalone offline application on mobile devices
- **Hybrid retrieval:** combine TF-IDF keyword search with semantic embeddings for best results
- **Try the CAG approach:** compare with the [local-cag sample](#) to see which pattern suits your use case

Clone the RAG sample, swap in your own documents, and have an offline AI agent running in minutes. Or compare it with the CAG approach to see which pattern suits your use case best.

[Get the RAG Sample](#) [Get the CAG Sample](#)

Summary

Building a local RAG application does not require a PhD in machine learning or a cloud budget. With Foundry Local, Node.js, and SQLite, you can create a fully offline, mobile-responsive AI agent that answers questions grounded in your own documents.

The key takeaways:

1. **RAG is ideal for scalable, dynamic document sets** where you need fine-grained retrieval with source attribution. Documents can be added at runtime without restarting.
2. **CAG is simpler** when you have a small, stable set of documents that fit in the context window. See the [local-cag sample](#) to compare.
3. **Foundry Local** makes on-device AI accessible: native SDK bindings, in-process inference, automatic model selection, and no GPU required.
4. **TF-IDF + SQLite** is a viable vector store for small-to-medium collections, with sub-millisecond retrieval thanks to inverted indexing and in-memory caching.
5. **Start simple, iterate outwards.** Begin with RAG and a handful of documents. If your needs are simpler, try CAG. Both patterns run entirely offline.

Clone the repository, swap in your own documents, and start building. The best way to learn is to get your hands on the code. This project is open source under the MIT licence. It is a scenario sample for learning and experimentation, not production medical or safety advice.

[local-rag on GitHub](#) · [local-cag on GitHub](#) · [Foundry Local](#)

Updated Mar 13, 2026

VERSION 1.0

AI

AI FOUNDRY

DEVELOPER

FOUNDRY LOCAL

GENAI

JAVASCRIPT

LEARNING

PERFORMANCE

RAG

SLM

...

 2

 Comment



Lee Stott MICROSOFT

Joined September 25, 2018

[View Profile](#)



[Microsoft Developer Community Blog](#)
Follow this blog board to get notified when there's new activity



No Comments
Be the first to comment



Enjoying the article? Sign in to share your thoughts.

Sign in

Share this page

[in](#)[f](#)[X](#)[r](#)[b](#)[RSS](#)[Email](#)

What's new

- Surface Pro
- Surface Laptop
- Surface Laptop Studio 2
- Copilot for organizations
- Copilot for personal use
- AI in Windows
- Explore Microsoft products
- Windows 11 apps

Microsoft Store

- Account profile
- Download Center
- Microsoft Store support
- Returns
- Order tracking
- Certified Refurbished
- Microsoft Store Promise
- Flexible Payments

Education

- Microsoft in education
- Devices for education
- Microsoft Teams for Education
- Microsoft 365 Education
- How to buy for your school
- Educator training and development
- Deals for students and parents
- AI for education

Business

- Microsoft AI
- Microsoft Security
- Dynamics 365
- Microsoft 365
- Microsoft Power Platform



Small Business

Developer & IT

- Azure
- Microsoft Developer
- Microsoft Learn
- Support for AI marketplace apps
- Microsoft Tech Community
- Microsoft Marketplace
- Marketplace Rewards
- Visual Studio

Company

- Careers
- About Microsoft
- Company news
- Privacy at Microsoft
- Investors
- Diversity and inclusion
- Accessibility
- Sustainability



Your Privacy Choices

Consumer Health Privacy

[Sitemap](#)

[Contact Microsoft](#)

[Privacy](#)

[Manage cookies](#)

[Terms of use](#)

[Trademarks](#)

[Safety & eco](#)

[Recycling](#)

[About our ads](#)

© Microsoft 2026